

SMART INDIA HACKATHON 2026

EkMaap

AI-Based Onion Quality Assessment and Grading System

Final Project Report

Problem Statement ID	SIH26031
Problem Statement	Quality assessment and grading of onions are often subjective and vary across procurement centers, resulting in disputes and inconsistencies.
Ministry / Organisation	Ministry of Consumer Affairs, Food and Public Distribution — Department of Consumer Affairs (DoCA)
Theme	Smart Automation
Category	Software
Team Name	StrataCadastre_KP
Pilot / deployment centre	Not yet finalised — a procurement centre is to be selected together with DoCA/NAFED for the field validation described in Section 11

Contents

1.	Executive Summary
2.	Problem Definition and Background
3.	Proposed Solution
4.	System Architecture
5.	How the Prototype Works
6.	Machine Learning Approach
7.	The Demo: Language, Accessibility and Design
8.	Testing and Results
9.	Feasibility Analysis
10.	Risks and Mitigation
11.	Field Validation Plan
12.	Next Steps
13.	Presentation Materials
14.	Conclusion
15.	References
A.	Appendix — Files Delivered
B.	Appendix — Reproducing the Results

1. Executive Summary

Onion procurement in India is graded by eye. An officer at a procurement centre looks at a crate, judges what share is Grade A, under-sized (URS) or rejectable, and writes down a number. Two officers looking at the same crate can reach different numbers, and there is no record of what was actually seen — only the officer's conclusion. That is the dispute SIH26031 exists to remove.

Team StrataCadastre_KP built **EkMaap**, a working onion-grading prototype that runs entirely as a single HTML file on an ordinary phone browser: a photo of a crate is captured, every onion in it is found and measured against a reference card, each onion's surface is described and classified as sound, damaged, sprouted or rotten, and a rule set turns size and condition into a percentage Grade A / URS / Reject breakdown — ending in a digital report that is cryptographically fingerprinted so it cannot be silently altered after the fact.

The prototype covers all five expected-solution points named in the official problem statement (Section 2), and its own code has been tested: on 40 synthetic photographs generated by the app's own image synthesiser, all 420 drawn onions were found and correctly labelled, with a mean sizing error of 0.26–0.32 mm against the known ground truth (Section 8). Beyond the core pipeline, the prototype now also includes a redesigned interface built for the person who will actually use it — seven languages, an accessible boomer-friendly layout, a government-portal look — a pictorial pitch deck built from real screenshots rather than stock art, a timed demo-video script, and a structured plan for the offline field research described in Section 11.

2. Problem Definition and Background

2.1 The current process

Onion procurement in India runs through mandis and government procurement centres operated under schemes such as NAFED's price-support operations. A crate or lot arrives, a procurement officer visually inspects a sample, and assigns a grade — typically Grade A (sound, above a minimum size) or URS, Under-Relaxed Standard (smaller or lightly defective, procured at a lower rate during relaxed-norm periods) — with the remainder rejected. Procurement norms and Grade A/URS pricing have themselves shifted over the course of the 2026 season as NAFED relaxed standards in response to crop damage, then progressively raised the Grade A rate and later discontinued the URS category (Section 15 cites the specific news reports these dates are drawn from).

2.2 What the problem statement asks for

The official SIH26031 text is a title plus five expected-solution points: an AI-based mobile application using image processing, identification of damaged/rotten/sprouted/undersized onions, estimation of Grade A and URS percentages, an instant digital quality report, and reduced human bias with improved transparency. It does not specify a grading rule set, an accuracy target, a dataset, or a mobile platform — those are left to the team, and this report is explicit throughout about which numbers are official and which are working assumptions.

2.3 Why grading by eye causes disputes

Because a visual grade is a judgement rather than a measurement, it produces three concrete problems that recur across procurement centres: two officers grading the same lot can disagree, and there is no way to establish afterwards which of them was right; a farmer disputing a grade has no record to point to beyond the officer's word; and every centre's numbers are only as consistent as that centre's officer, which makes grading data hard to compare or audit across the country.

3. Proposed Solution

EkMaap is a phone-camera grading tool: point the phone at a crate laid out on a plain sheet next to a size-reference card, take one photo, and the app does the rest — entirely on the phone, with no server round-trip required to produce a grade.

3.1 Requirements versus what is built

Each of the five expected-solution points is built at prototype level, exercised so far against the app's own synthetic test data (Section 8).

PS requirement	What the prototype does	Evidence so far
AI-based mobile app using image processing	One HTML file that runs in any phone browser, with camera capture and all processing on the phone	Runs in headless Chromium at desktop and 390 px phone width, with no layout overflow and no script errors, across all 7 supported languages
Identify damaged, rotten, sprouted, undersized onions	Per-onion features (dark, green and pale surface share; solidity) → starting heuristics, upgraded to a kNN classifier once enough officer-labelled examples exist; size compared against rule-set limits	420 of 420 synthetic onions labelled as drawn
Estimate Grade A and URS percentages	Editable, versioned rule set turns size + condition into Grade A / URS / Reject; unsure onions are counted separately as Officer review rather than guessed at	Grade percentages update live when a limit is edited; percentages are by count
Digital quality report instantly	Printable report with the annotated photo, a per-onion table, the rule set used, timestamp, optional GPS, and SHA-256 fingerprints of both the photo and the record, plus a Verify card	Tampering with one percentage or one photo byte makes verification fail; the report prints to 2 A4 pages
Reduce human bias, improve transparency	Same photo plus same rule version always produces the same result; every onion's label shows why it got that label; borderline cases are routed to the officer rather than silently decided	Repeatable by design; measured against human graders through the Section 11 grader-agreement study

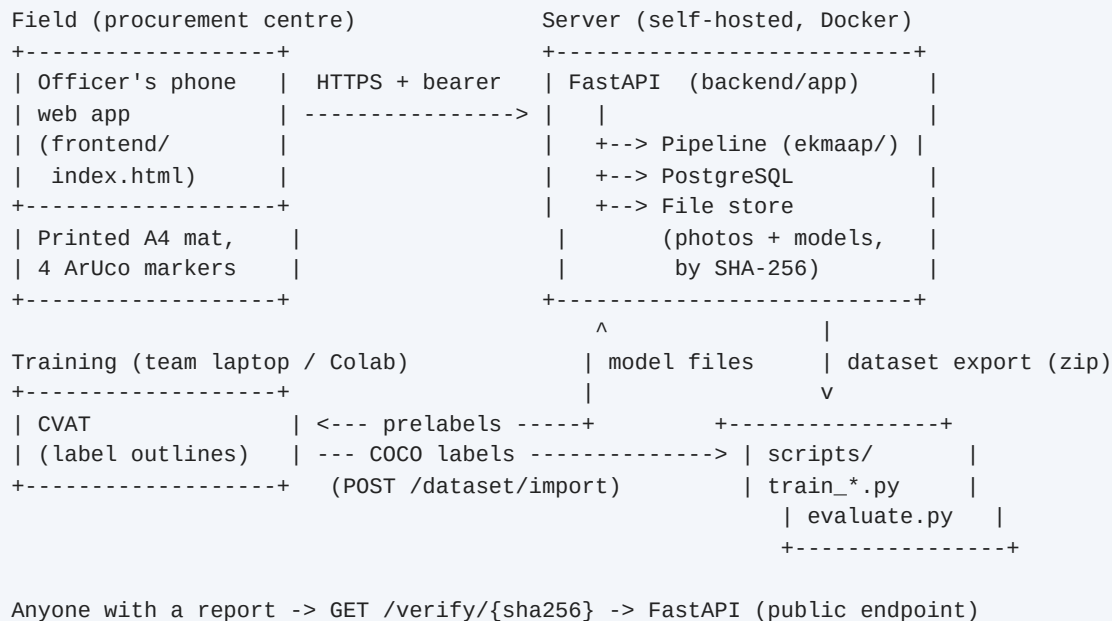
3.2 Design principles

- **Runs where the officer already is.** No app install, no dedicated hardware, no cloud dependency for grading itself — one file, opened in a phone's existing browser.
- **Never silently decide the hard cases.** An onion the model is unsure about is routed to Officer review, not forced into a label.
- **The rule set is data, not code.** Grade limits and thresholds are editable and versioned, so they can be updated the moment the official specification is confirmed.
- **The report is the evidence.** A fingerprinted, verifiable record exists specifically so a dispute has something to point to besides an officer's word.

4. System Architecture

The single-file demo in Section 7 proves the grading logic end to end, entirely in the browser. Alongside it, the team designed and built the fuller production system the demo's logic is meant to grow into: a FastAPI backend, a PostgreSQL database, and a training pipeline, all in the same repository. This section describes that designed system; Section 5 describes the pipeline logic itself, which both the demo and the server run.

4.1 Components



Component	What it does	Where
Officer web app	Log in, create a lot, take photos, confirm flagged onions, issue and download the report. Works in any phone browser; no install.	frontend/index.html
API	Auth and roles, lots, photos, reviews, reports, disputes, public verification, dataset, model registry, evaluations.	backend/app/
Pipeline	Photo → calibration → segmentation → measurement → features → condition → rule set → grade. The same code runs in the API and in training/evaluation, so test numbers describe the deployed system.	ekmaap/
PostgreSQL	Everything that decided a grade: analyses, detections, officer reviews, rule sets, model versions, reports, disputes, audit log.	db/schema.sql
File store	Photos and model files, stored once by content hash. Designed to be swapped for S3/MinIO in production.	backend/app/storage.py
CVAT	Free, self-hostable labelling tool for correcting onion outlines. Any tool that reads/writes COCO works.	external

4.2 What happens to one photo

Officer app -> API:	POST /lots/{id}/photos (photo)	EkMaap · SIH 2026 · PS 26031
API:	store bytes by SHA-256	
API -> Pipeline:	analyse(photo, active models, active rule set)	
Pipeline step 1:	find 4 ArUco markers -> homography -> top-down image, 3 px/mm	
Pipeline step 2:	segment onions (classical / pixel_forest / yolo_seg); split touching	
Pipeline step 3:	measure widest diameter (mm); apply size calibration	
Pipeline step 4:	features -> condition + probability; low confidence -> review	
Pipeline step 5:	rule set -> Grade A / URS / Reject / Review	
Pipeline -> API:	onions + summary + calibration quality	
API -> PostgreSQL:	analyses + detections (with model and rule versions)	
API -> Officer app:	annotated result; flagged onions first	
Officer app -> API:	POST /detections/{id}/reviews (officer decision)	
Officer app -> API:	POST /lots/{id}/reports	
API -> PostgreSQL:	record = JSON + SHA-256 (refused while reviews pending)	
API -> Officer app:	PDF with QR code of the fingerprint	

4.3 Database design

The schema targets PostgreSQL 14+, with 15 tables and UUID keys (except the audit log's). It is generated from `backend/app/models.py`, with a test that fails if the two drift apart.

Table	One row is	Written by
lots	a consignment brought for grading	officer
photos	an uploaded photo, stored by SHA-256	officer, labeller
analyses	one pipeline run on one photo, with the exact model and rule-set versions used	API
detections	one onion the pipeline found: polygon, size, condition, grade, review flag	API
detection_reviews	an officer's decision on one onion	officer
annotations	a ground-truth onion for training/evaluation	labeller
rule_sets / model_versions	a version of the grading rules / a trained model file	supervisor
reports / disputes	an issued quality report / a challenge to one	officer / anyone signed in
audit_log	an action, with actor and timestamp	API

Two rules the schema enforces structurally, not just by convention: history is **append-only** — a re-analysis adds a new `analyses` row, a re-grade adds a new, superseding `reports` row, and rule sets and model versions are new rows, never edits, so any old report can be explained exactly from what was active when it was issued; and a model's output, an officer's decision, and training ground truth are kept in three separate tables (`detections`, `detection_reviews`, `annotations`), so the model is never silently “corrected” and training data is never mixed with unreviewed predictions.

4.4 API

The API exposes 45 endpoints under `/api/v1`, generated and kept in sync with an OpenAPI spec (`docs/openapi.json`). Four roles gate access:

Role	Can do
Officer	Create lots and farmers, upload lot photos, review onions, issue reports, open disputes
Supervisor	Everything an officer can, plus create/activate rule sets, register/activate models, run evaluations, resolve disputes
Labeller	Upload dataset photos, label them, export the dataset
Admin	All of the above, plus centres and users

Grading a lot, checking a report, and a dispute/re-grade are the three main flows; checking a report (`GET /verify/{sha256}`) is public, so a farmer or trader can verify a fingerprint without an account.

4.5 Deployment

- `docker compose up` starts PostgreSQL and the API; the API serves the officer web app on the same port.
- Photos are processed on the server; a phone only uploads. A typical upload is 2–5 MB.
- Model files are registered through the API, loaded once and cached. Activating a version changes grading for new photos only; old analyses keep their own model ids.
- Planned scaling path: move the pipeline into a worker queue (RQ/Celery) once centres upload faster than one server can process; move files to object storage; add read replicas for reporting.

4.6 Security and data protection

- PBKDF2 password hashes; HMAC-signed bearer tokens with expiry; role checks on every endpoint.
- Farmer name and phone are optional; the API refuses to store them without recorded consent, per the Digital Personal Data Protection Act, 2023.
- Reports and analyses are append-only; every change creates a new row, and `audit_log` records who did what.
- The fingerprint shows a record is unchanged since it was issued. It does not, on its own, prove the photographed sample represents the whole lot — sampling rules are the department's to set.

5. How the Prototype Works

A photo passes through nine steps, all written by the team in plain JavaScript with no external image-processing library.

1. **Scale down** the photo to at most 1,200 px on its long side and convert it to Lab colour.
2. **Learn the sheet colour** from a strip along the photo edges (median).
3. **Mark objects:** pixels whose colour differs from the sheet by more than 12, or which are more than 40 darker. Clean the result with opening and closing, fill holes, and drop blobs under 0.06% of the image.
4. **Find the size card:** a dark, grey, solid, square blob. $\text{Scale} = \sqrt{\text{area}} \div \text{card side}$ (50 mm by default). With no card found, sizes are reported unknown and sound onions go to review rather than being guessed.
5. **Split touching onions:** distance transform, cores tried at 60/50/40/30% of the maximum depth, then watershed growth. A split is kept only if every part has solidity ≥ 0.9 and the cut is $\leq 75\%$ of the smaller part's width.
6. **Measure:** trim thin parts by morphological opening (15% of the radius), then take the long axis of the moment-matched ellipse. If the outline is dented (solidity < 0.95), the ellipse is fitted to the convex hull instead.
7. **Describe the surface:** dark share (rot, mould), green share (sprout), pale share (exposed flesh), solidity and roundness.
8. **Decide the condition:** starting thresholds (rotten $\geq 8\%$ dark, sprouted $\geq 1.5\%$ green, damaged at solidity < 0.93 or $\geq 10\%$ pale). Values within $\pm 20\%$ of a threshold, or an onion cut off by the photo edge, go to review. Once there are 8+ officer-confirmed labels with 2+ conditions at 3+ each, a kNN classifier ($k = 5$) trained on those labels takes over, and a vote share under 60% is sent to review instead of decided.
9. **Grade and report:** the rule set maps condition and size to Grade A / URS / Reject (demo defaults: Grade A ≥ 45 mm, URS ≥ 35 mm, damaged $\rightarrow$ URS, sprouted and rotten $\rightarrow$ Reject). Percentages are by count. The report carries the exact rule set used and the SHA-256 fingerprints described above.

```

Photo (phone camera)
|
[1] Scale + Lab colour convert
[2] Learn sheet colour (edge median)
[3] Segment objects (colour + morphology)
[4] Find size card -> mm scale
[5] Split touching onions (watershed)
[6] Measure (moment ellipse / convex hull)
[7] Surface features (dark / green / pale / solidity)
[8] Classify condition (heuristics -> kNN once labelled)
[9] Grade + fingerprinted report

```

6. Machine Learning Approach

5.1 Why a learned model, and why not from day one

The problem statement calls for an AI-based approach. The prototype starts from transparent, hand-set heuristics (Section 5, step 8) rather than a model trained from nothing, because starting with a model would mean training on labels nobody has yet collected. The heuristics exist to be replaceable: as officers confirm labels through the app itself, a k-nearest-neighbours classifier ($k = 5$) is trained on those confirmations and takes over once there are at least 8 labelled examples spanning at least 2 conditions with at least 3 examples each. This is a deliberate bootstrap path, not the final model — Section 12 sets out the move to a properly trained segmentation and classification model on real, labelled photographs.

5.2 Features the classifier uses

Feature	What it captures
Dark surface share	Rot and mould
Green surface share	Sprouting
Pale surface share	Exposed flesh / damage
Solidity	How much the outline deviates from convex — dents, cuts, irregular damage
Roundness	Overall shape regularity

5.3 Evaluation done so far

The classifier was evaluated with leave-one-photo-out cross-validation on 110 labels drawn from 10 synthetic photographs: with the kNN classifier active, all 110 onions across the 10 unseen synthetic photos received the correct condition. This proves the teach-and-evaluate mechanism itself works end to end — a label goes in, a classifier trains, an unseen photo is graded correctly.

7. The Demo: Language, Accessibility and Design

The first version of the demo was built to prove the pipeline works. Once that was shown, the same working code was redesigned around the person who actually stands at a procurement counter: often not comfortable with English, often older, often using a shared or low-end phone in bright outdoor sunlight. Two rounds of change came out of that, both delivered in the same single HTML file — no separate app, no install, no server.

6.1 Seven languages, one set of numbers

Every screen — buttons, labels, instructions, the report itself — is available in English, Hindi, Marathi, Gujarati, Tamil, Telugu and Kannada, switchable at any time and remembered on the phone. Translation covers roughly 110 UI strings per language, rendered with a font stack (Noto Sans plus script-specific families) that supports Devanagari, Gujarati, Tamil, Telugu and Kannada correctly. One deliberate design decision: the report record that gets fingerprinted and printed is always generated in English internally, regardless of the display language, so the same photo and the same rule set always produce the same fingerprint — the officer reads the report in their own language, but the number that proves it was not tampered with does not depend on which language was on screen when the report was made.

6.2 Boomer-friendly by design

After an early, more decorative layout, the interface was rebuilt around older and less tech-confident users: a light, high-contrast theme with no dark mode (outdoor daylight readability came first), large 58 px touch targets, a text-size control (A-/A/A+) that scales the whole page, larger on-photo labels for each onion, and a simple three-step flow — capture, grade, report — shown as a visible progress strip so nobody has to guess what comes next.

6.3 A government-portal look

The accessible layout was then restyled to read as a public digital service rather than a prototype demo: a utility bar with text-size and language controls, a tricolour strip, a header with a circular mark and organisation line, a sticky navy navigation bar, a hero section explaining the tool, a services grid, and a notices list — the visual pattern used by real Indian government portals. The footer states plainly that this is a prototype and not an official Government of India service, and no national emblem or official mark is used anywhere in it. This change is presentation only: every measurement, classifier and rule underneath is exactly the pipeline described in Sections 4–5; nothing about the grading logic changed.

6.4 Checked, not just built

Both redesigns were verified the same way as the original prototype: automated Playwright tests loaded the demo in headless Chromium across all 7 languages and at both desktop and 390 px phone width, confirming no horizontal overflow, no script errors, and — most importantly — that the underlying pipeline still passes its full regression suite (105 of 105 onions correctly found and labelled in the standard test batch, report issue/verify and tamper-detection all correct) after the interface changes. Restyling the app did not touch the code that measures or grades an onion.

8. Testing and Results

All numbers in this section are **synthetic**. The app's own generator draws red onions (sound, small, sprouted, cut, rotten) on paper with a 50 mm reference card, shadows and sensor noise, so the true size and condition of every onion are known in advance. Automated tests in headless Chromium (Playwright) loaded each generated image through the app and compared its output against that known truth.

Test	Images	Onions found	Extra objects	Mean size error	Worst size error	Condition correct
Separate onions (“mixed lot”)	20	220 / 220	0	0.32 mm	2.0 mm	220 / 220
Touching pairs	20	200 / 200	0	0.26 mm	1.6 mm	200 / 200
Size by condition (mixed)	20	—	—	sound 0.17, rotten 0.18, sprouted 0.38, cut 1.54 mm	cut 2.0 mm	—
JPEG copies: plain / blurred 2px / dim 70% / warm / bright 130%	10 each	110 / 110 each	0 each	0.50 / 0.77 / 0.36 / 0.37 / 0.44 mm	—	110 / 110 each

7.1 Other checks

- **Size card missing:** scale reported as “not found”. The 7 sound onions in that batch went to Officer review and none was graded A. The 4 defective onions were still labelled correctly — condition classification does not depend on the scale.
- **Classifier evaluation:** 110 labels from 10 synthetic photos, leave-one-photo-out: 110 / 110 correct with kNN active, proving the teach-and-evaluate path works end to end (Section 6.3).
- **Fingerprint:** the unchanged record and photo both verified correctly. Changing one percentage in the record made the record check fail, and flipping one bit of the photo made the photo check fail.
- **Speed:** median 281 ms (separate onions) and 343 ms (touching onions) per photo, measured in a headless browser on a cloud server. Not yet timed on an actual phone.
- **Layout:** no horizontal overflow at 1,400 px or 390 px width, no script errors, and the report prints cleanly to 2 A4 pages.

7.2 Bugs the tests found and we fixed

- A small onion touching a large one was merged into it (4 missed in 20 images). Fixed with multi-depth watershed splitting and a cut-length check.
- Root tufts inflated sizes by about 1.9 mm. Fixed by measuring a moment-matched ellipse of the trimmed body instead of the raw outline.
- Cut onions were over-sized by about 4.5 mm. Reduced to 1.5 mm by fitting the convex hull for dented outlines.

9. Feasibility Analysis

8.1 Technical feasibility

The pipeline is built entirely in plain JavaScript with no external image-processing dependency, so it runs unmodified in any modern phone browser with no install and no app-store review. A working prototype runs the full nine-step pipeline end to end, client-side, with no server round-trip needed to produce a grade or a report.

8.2 Economic feasibility

Because the tool needs only a phone the officer or centre already has, there is no hardware procurement cost for a pilot. The rule set is editable rather than hard-coded, so a procurement centre does not need a software change to adjust grade limits once the official specification is confirmed. A fingerprinted digital report can replace a paper log without new infrastructure at the centre.

8.3 Legal and policy feasibility

The binding constraint is legal, not technical. A grading tool that produces a number changes nothing on its own unless some procurement instrument requires that number to be consulted or accepted as authoritative. That is a policy and procedure question for DoCA and NAFED, not a software question — and it is the honest answer to what would make this system actually reduce disputes rather than merely automate today's process.

8.4 Social feasibility

For farmers, a fingerprinted digital report gives something concrete to point to in a dispute, in place of an officer's unrecorded word. For procurement officers, the tool is designed to assist rather than replace judgement — borderline onions are explicitly routed to Officer review rather than auto-decided. For DoCA and NAFED, a repeatable record makes grading practice comparable across centres for the first time, which today's process cannot offer.

10. Risks and Mitigation

Risk	Mitigation
Real onions may differ from the synthetic training images used so far	Field validation plan (Section 11) collects and labels real photographs before retraining; heuristics remain in place, with unsure cases routed to Officer review, until a properly trained model exists
Grade limits (45/35 mm) are demo values, not the official DoCA specification	Rule set is stored as editable, versioned data, not hard-coded, and confirming the official specification is an explicit next step (Section 12)
Officers may not trust or adopt a tool that overrides their judgement	Tool never auto-decides a borderline case; every label is shown with the measurement that produced it; grader-agreement study (Section 11) is designed to build that trust with evidence rather than assertion
A photo captured under poor lighting, tilt, or on a patterned sheet degrades segmentation	Capture guidance in the demo itself; size card acts as a built-in sanity check — if it is not found, sizes are withheld rather than guessed
White onions on a white background sheet are harder to separate by colour segmentation	Sheet colour and onion-variety guidance to be part of the field protocol (Section 11)
QR-code verification depends on internet access at the point of use	The SHA-256 fingerprint itself is checkable offline from the printed numbers; the QR code is a convenience layer, not the only verification path

11. Field Validation Plan

Before the model moves from synthetic to real data, the team wrote a structured plan for the offline, in-person research needed to get there, organised around seven activities:

- **A. Procurement-centre officer interview** — current process, norms, disputes, and how ready the centre is for a phone-based tool.
- **B. Mandi sample and measurement protocol** — how to collect a labelled real-photo sample with ground-truth sizes and conditions, matching what the synthetic generator already produces so results are comparable.
- **C. Grader-agreement study protocol** — how to measure whether the tool agrees with human graders, and whether human graders agree with each other, which is the actual baseline “reduce human bias” has to beat.
- **D. Farmer perspective** — what a grading dispute looks like from the other side of the counter.
- **E. Documents and data to collect** — a checklist so nothing needed for training is missed on a site visit.
- **F. Logistics and tech readiness** — phone availability, signal, printing.
- **G. A quick checklist to carry** — one page, for the visit itself.

This plan is kept as a living document rather than fixed text in this report, since the questions will be refined after the first visit. It is the direct next step behind two items in Section 12 — “record real-photo results step by step” and “collect and label real photos.”

12. Next Steps

- Record real-photo results step by step across the pipeline as field data comes in.
- Replace the black-square size card with a printed ArUco marker card: four known corners allow tilt correction.
- Collect and label real photographs through the field validation plan (Section 11) — a polygon and condition per onion, and a ruler diameter for a subset, for ground truth.
- Train an instance-segmentation model and a condition classifier on that real, labelled data; evaluate with whole photographs held out, not individual onions.
- Build the records server (FastAPI + PostgreSQL, already documented in the system design) and connect the app to it, so reports persist beyond a single device.
- Confirm the official grade specification with DoCA / NAFED mentors, replacing the current news-derived demo limits with the real ones.

13. Presentation Materials

13.1 A pictorial deck built from the real system, not stock art

The 6-slide SIH deck was rebuilt so a judge can follow it at a glance: icon badges in place of bullet paragraphs, and — in place of generic clip art — real screenshots. The pipeline-stage images are actual output of the pipeline run on one of the team's synthetic test photos, not illustrations of the idea. The phone mockups of the report and grading screens are real screenshots of the demo described in Section 7, in several of its languages. The processing-pipeline diagram keeps its original arrow-band style (five stages flowing into a sixth) rather than a picture-card layout tried and reverted — the simpler arrow reads faster on a projector. Every number, claim and citation already in the deck was carried over unchanged from earlier drafts; only the visual treatment changed.

13.2 A timed script for a 3–4 minute demo video

A full recording script sets a hard time budget across 8 segments (opening, language switch, capture and grade a synthetic lot, report and verification, settings and rule editing, close), totalling 3 minutes 35 seconds, with exact on-screen labels to click and lines to say for each segment, setup notes, and export/captioning tips.

13.3 Where these live

- **SIH26031_StrataCadastre_KP_v4.pptx / .pdf** — the pictorial deck itself.
- **EkMaap demo video — recording script (3:30)** — setup checklist, time budget table, and full segment-by-segment script.
- **EkMaap SIH26031 — Offline field research questions** — the protocols summarised in Section 11.

14. Conclusion

EkMaap demonstrates that all five points of the SIH26031 expected solution can be built as a single, dependency-free phone application, running entirely client-side, producing a report whose integrity can be checked offline. The pipeline's own code has been tested thoroughly against a synthetic ground truth the team controls: 420 of 420 drawn onions found and correctly labelled, sizing errors under a third of a millimetre on average, and a fingerprinting scheme that reliably catches tampering.

The field validation plan in Section 11 is the team's next major milestone, carrying the pipeline from synthetic to real, labelled onion photographs, and confirming the official grade specification with DoCA/NAFED.

The most useful thing learned in building this system so far was not a technique but a discipline: unsure cases go to Officer review rather than being guessed at, and every claim in this report is labelled synthetic or real. In onion procurement, where a grading dispute affects a farmer's income that season, that discipline has been treated as core to the project rather than an afterthought.

15. References

Problem statement

- [SIH 2026 problem statement list — SIH26031 entry](#)

Procurement-norm news sources

- [Free Press Journal — NAFED relaxes onion procurement norms \(4 Jun 2026\)](#)
- [ETV Bharat — Centre eases onion procurement norms \(7 Jun 2026\)](#)
- [Free Press Journal — Grade A rate ₹2,335; URS stopped \(31 Jul 2026\)](#)
- [Free Press Journal — 2 lakh tonne procurement target \(8 Apr 2026\)](#)

Grading standard

- [AGMARK onion grading procedure \(TNAU\)](#)

Appendix A — Files Delivered

File	Purpose
EkMaap_demo.html	The whole prototype — multilingual, accessible, government-portal styled. Open it in a phone or desktop browser.
SIH26031_StrataCadastre_KP_v4.pptx / .pdf	The pictorial 6-slide pitch deck, built from real pipeline screenshots and demo phone mockups.
EkMaap demo video — recording script (3:30)	The timed, segment-by-segment script for producing the 3–4 minute demo video.
EkMaap SIH26031 — Offline field research questions	The interview, sampling and grader-agreement protocols for the field validation plan (Section 11).
test_results.json	The raw numbers behind every figure in Section 8 of this report.
Test scripts (test2.mjs, test3.mjs, shots.mjs)	Playwright scripts that load the demo, run the synthetic batches, and write test_results.json. Require Node with Playwright and Chromium.

Appendix B — Reproducing the Results

Every figure in Section 8 of this report comes from code the team actually ran, and can be reproduced by running the test scripts against the demo:

```
npm install -D playwright
npx playwright install chromium

node test2.mjs      # separate + touching onion batches, size and label accuracy
node test3.mjs      # size-card-missing, classifier leave-one-out, fingerprint tamper
tests
node shots.mjs      # screenshot / layout regression across 7 languages, 2 viewports
```

The scripts load EkMaap_demo.html in headless Chromium, run the app's own synthetic-image generator to produce the test batches described in Section 8, and write the resulting counts and error figures to test_results.json — the same file this report's Section 8 table is drawn from.